If for some reason you don't like how Django's template language works, you can plug in another template backend, or just import some other Python library to handle this bit for you. Of course in this case it might become harder for you to integrate with other parts of Django that use its own template language, such as the admin panel.

So without further ado, let's introduce the syntax of Django's template language, the final piece in our understanding of Django.

The Django Template Language

The first thing to know about Django's temple language is that it doesn't know, or care if the output it is generating is actually HTML or not, all it sees is text. While this means that you can generate non-HTML, but text based output using these templates, it also means that you are fully responsible for ensuring that you are generating valid HTML, which can be harder when you have non-HTML template code in the same file.

Another thing you should know before diving in is, where do you keep these templates anyway? Since templates are associated your 'cms' app, you will need to place them in a folder called templates in your 'cms' app directory. Here too it is important to note the conventions Django applications tend to use. Since out application is named 'cms' we need to create a directory called 'cms' within the templates directory for Django to automatically find some of our templates.

In the previous chapter, before we showed how to use templates, we just kept our HTML code in a Python string; keep that code in mind while we present to you the Django template code to generate the same output:

```
<html>
<head><title>{{ article.title }}</title></head>
<body>
<h1>{{ article.title }}</h1>
<p>Author: {{ article.author }}</p>
<p>Category: {{ article.category }}</p>
{{ article.content }}
</body>
</html>
```

Notice how similar this is to the code we originally used! In fact the major difference seems to be that we are using double curly braces instead of single ones. If you flip back the page with the original code, you will see that while generating a response from this template we passed it a context object that